ENTERPRISE DEVOPS INFRASTRUCTURE

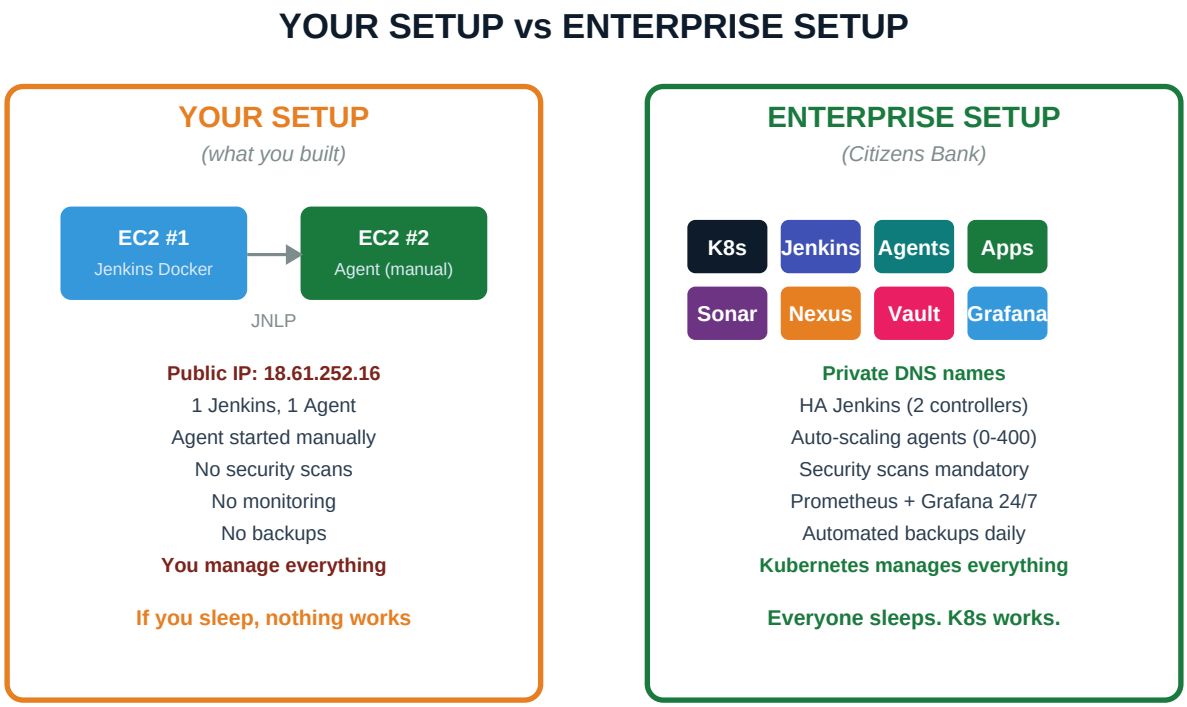
How Real Companies Run DevOps on Kubernetes

THE VISUAL GUIDE

7 Diagrams That Show the Complete Picture

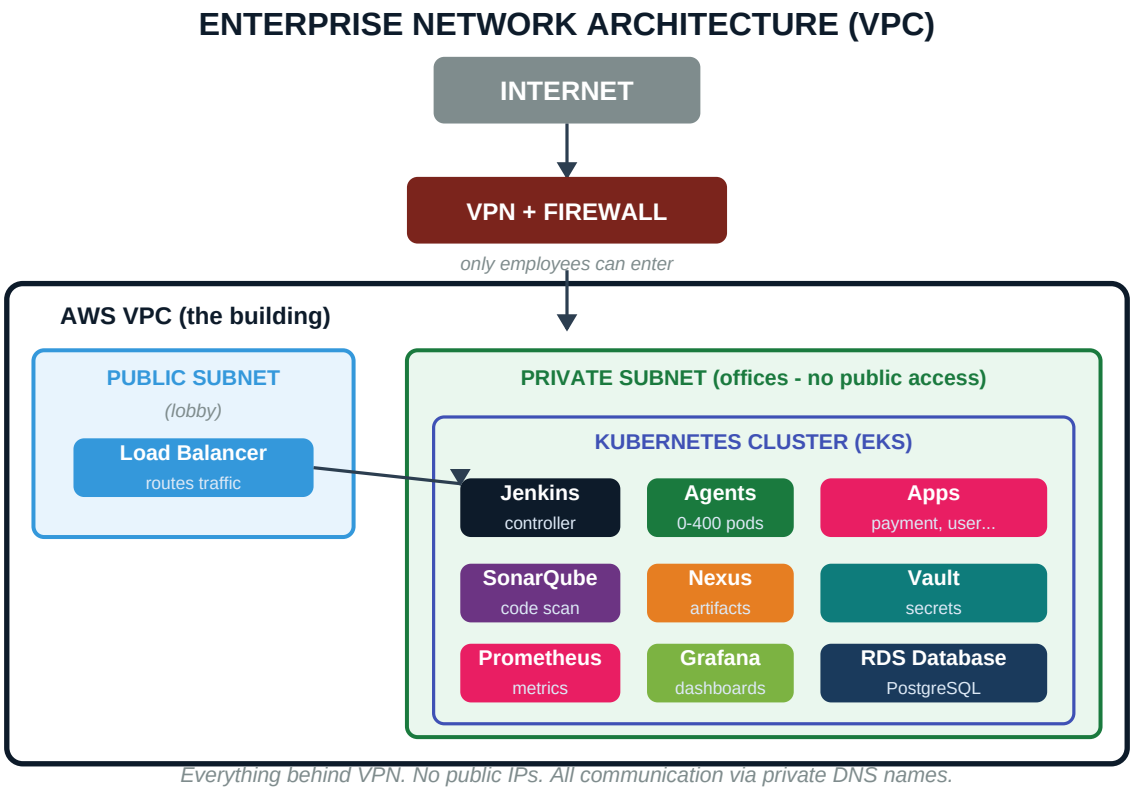
Your Setup vs Enterprise | VPC Architecture | Kubernetes Pods Build Flow | Deployment Pipeline | Self-Healing | 400 Jobs Solved

Diagram 1: Your Setup vs Enterprise



KEY INSIGHT: The architecture is the same — controller + agents. The difference is automation. Enterprise wraps everything in Kubernetes so nothing depends on a human being awake.

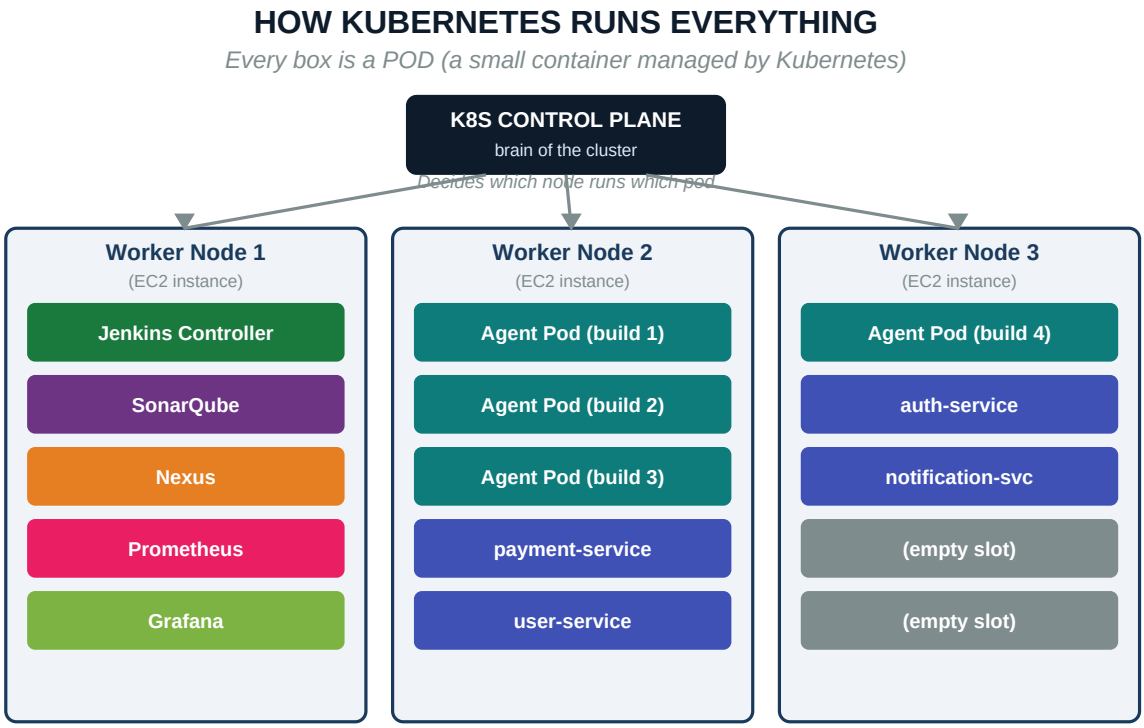
Diagram 2: Enterprise Network Architecture



Everything sits inside a VPC (Virtual Private Cloud) — like a building with walls. The Load Balancer is the only door to the outside. Everything else is in private rooms with no public access. Employees must use VPN to enter the building.

> **SECURITY DIFFERENCE:** Your Jenkins at http://18.61.252.16:8080 is on a public IP — anyone on the internet can try to access it. Enterprise Jenkins is behind VPN + private subnet + load balancer. Three layers of security before you even reach Jenkins.

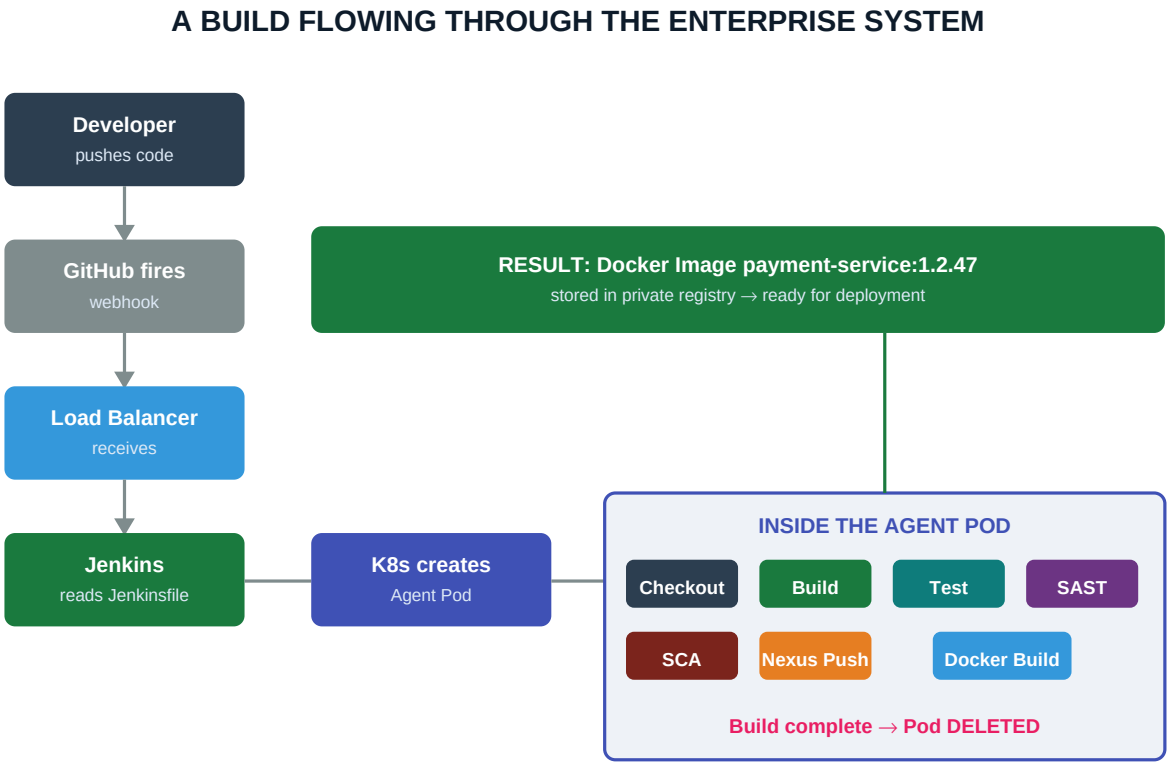
Diagram 3: How Kubernetes Runs Everything



Kubernetes has Worker Nodes (EC2 instances) and Pods (containers running on those nodes). The Control Plane decides which pod runs on which node. If a node dies, Kubernetes moves all its pods to other nodes automatically. Nobody SSHes into anything.

> **KEY INSIGHT:** Every tool — Jenkins, SonarQube, Nexus, Grafana, your applications — they are all pods. Kubernetes manages all of them identically. Crash? Restart. Overloaded? Scale. Node dies? Migrate. One system to rule them all.

Diagram 4: A Build Flowing Through the System

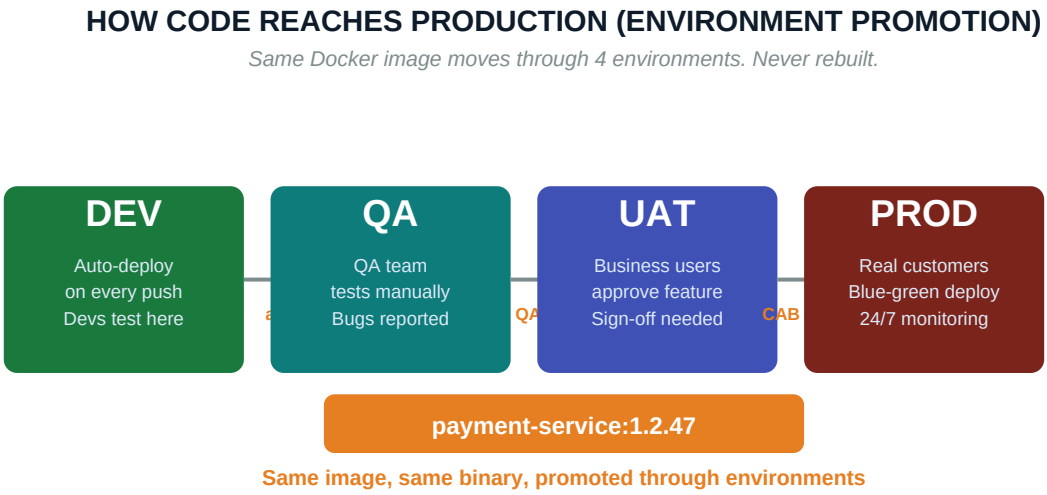


Total time: ~2 minutes | Agent pod existed for ~2 minutes | Cost: fractions of a dollar

When a developer pushes code, the webhook hits the Load Balancer, reaches Jenkins, Jenkins tells Kubernetes to create an Agent Pod. The pod runs all pipeline stages (build, test, scan, push). When done, the pod is deleted. The entire agent existed for just 2 minutes.

> **KEY INSIGHT:** Compare to your setup: you manually started agent.jar, it ran forever, used resources even when idle. Enterprise creates agents on demand and destroys them when done. Zero waste.

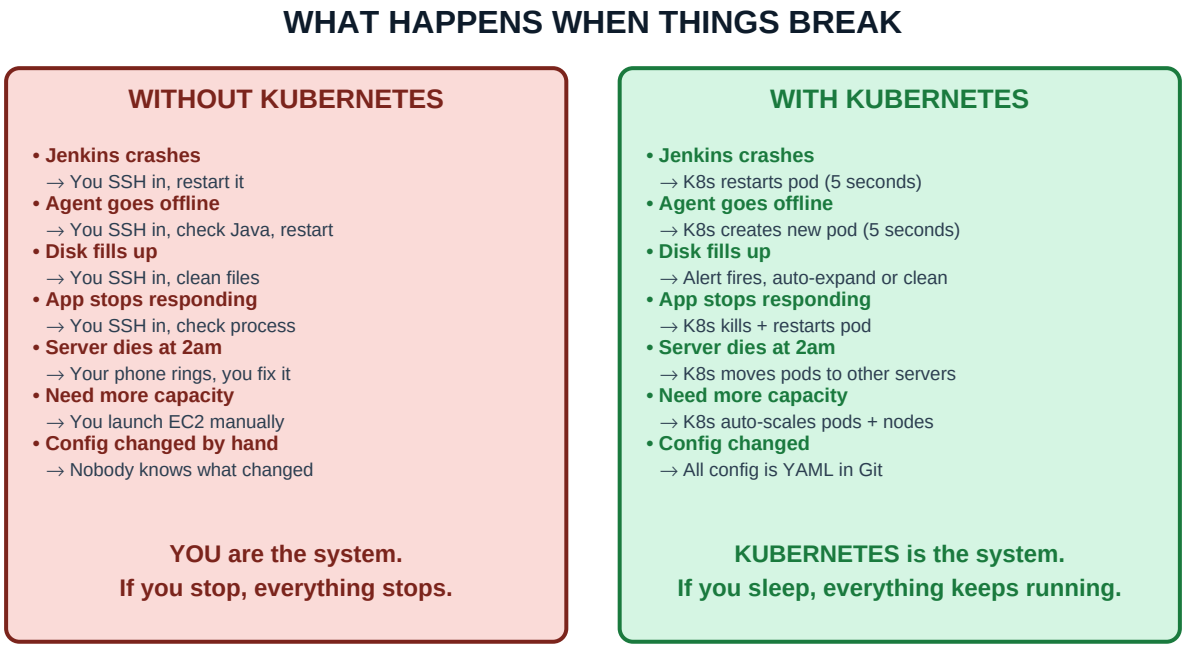
Diagram 5: How Code Reaches Production



The SAME Docker image travels through DEV → QA → UAT → PROD. It is never rebuilt. What was tested in DEV is exactly what runs in production. Each promotion requires passing a gate — automated tests, QA approval, or CAB approval.

> **KEY INSIGHT:** This is the "immutable artifact" principle. One build produces one image. That image is tested 4 times in 4 environments. Only after all gates pass does it reach real customers. If production breaks, switch back to the previous image in 30 seconds.

Diagram 6: Self-Healing — Without vs With Kubernetes

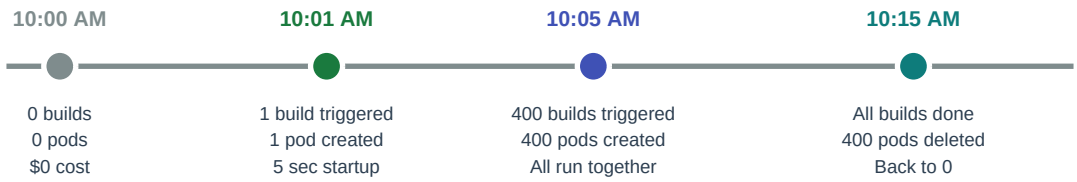


Without Kubernetes, every failure requires a human. With Kubernetes, most failures are handled automatically. This is why every large company uses Kubernetes — it replaces the human on-call for routine failures.

YOUR EXPERIENCE TODAY: You experienced this today: Jenkins agent crashed, you spent 4 hours fixing it manually. With Kubernetes, the pod would have been recreated in 5 seconds automatically. That 4-hour struggle becomes a 5-second automatic recovery.

Diagram 7: The 400 Jobs Problem — Solved

THE 400 JOBS PROBLEM — SOLVED BY KUBERNETES



Your setup: 400 jobs / 2 executors = 200 batches = HOURS to drain

K8s setup: 400 jobs = 400 pods = ALL run simultaneously = MINUTES to drain

This is the exact question that failed you at Citizens Bank. With your setup: 400 jobs wait in a queue for 2 executors. With Kubernetes: 400 pods created in seconds, all 400 builds run simultaneously, pods deleted when done.

> **KEY INSIGHT:** The interviewer was not asking about a hypothetical problem. This is their daily reality. 400+ builds per day across 200 developers. Kubernetes dynamic agents are the only way to handle this at scale.

One Sentence to Remember

Kubernetes is the operating system for your infrastructure.

Just like Ubuntu runs programs on one computer, Kubernetes runs containers across hundreds of computers.

Every tool you learn — Jenkins, Docker, Nexus, SonarQube — they all become pods inside Kubernetes. Kubernetes runs them, monitors them, restarts them, scales them.

Now you see the complete picture. Put this on your wall.